ADR 0006 — Trace format

Status: Proposed (pending Ronin discussion per RONIN_SYNC.txt Tier 1 #1) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, ADR 0010, docs/DEFERRED.md, src/baymax/telemetry/logger.py

---

Context

BAYMAX needs a trace format for agent runs that:

1. Captures the full lifecycle of a request — input arrival, task_file construction, validation, per-tool dispatch, recovery actions, final response
2. Is consumable by Ronin's eval harness when investigating scenario failures ("why did this scenario fail at step 3?")
3. Carries enough structure to be queried (e.g. "show me all runs where the model produced an `intent_error` on Qwen but succeeded on GPT-4")
4. Supports correlation across the agent → inference backend → tool adapter call chain
5. Doesn't require additional infrastructure (no separate collector daemon) for v1

The industry options:

- **OpenTelemetry (OTel)** — vendor-neutral standard, native span/trace/baggage model, requires a collector process (OTel Collector, Jaeger, Tempo) to receive and store spans
- **Custom JSON lines (ndjson)** — one JSON event per line, written to a file or stdout, read by trivial tooling
- **Plain text logs** — easy but unstructured; hard to query without grep+awk gymnastics
- **A database-backed event log** — clean queries but adds operational surface

Ronin's eval harness is the primary downstream consumer in v1. There's no production deployment, no separate dashboard, no SRE team waiting for traces.

Decision

v1: Custom JSON lines (ndjson). Each event is one line in a `.jsonl` file under `traces/<run_id>/<task_id>.jsonl`. Event shape:

```
{
  "ts": "2026-06-20T14:32:17.482Z",
  "trace_id": "tr_8c7f...",
  "task_id": "<run_id>.<scenario_id>.<phase_int>",
  "event": "tool.dispatch.start",
  "module": "baymax.core.executor",
  "payload": {
    "tool": "calendar.create_event",
```

```
    "arguments_redacted": { "title": "Meeting with Sarah", "start": "2026-06-21T09:00:00Z" }
  }
}
```

Event categories (initial set, extensible):

- `request.received` — user input arrives
- `task_file.constructed` — model produced a task_file
- `validation.passed` / `validation.failed` — tool-call contract check result
- `tool.dispatch.start` / `tool.dispatch.end` — adapter call boundary
- `recovery.triggered` — recovery handler invoked
- `clarification.emitted` — model asked the user for clarification
- `response.emitted` — final `AgentResponse` returned

OpenTelemetry trace format is **deferred to v2** (DEFERRED.md). Trigger: distributed tracing across multiple processes becomes useful (e.g., when state store is Redis per ADR 0007, or when eval runs are sharded across machines).

PII/auth redaction per ADR 0010 §redaction is applied at event-emission time. Fields matching `auth*`, `token*`, `key*`, `password*`, `secret*`, `authorization*` are replaced with `[REDACTED]`.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable in 6 months. Not interview-defensibility depth.]

- **WHY ndjson over OpenTelemetry in v1**: zero new infrastructure is the point. The eval harness reads a `.jsonl` file with `pandas.read_json(lines=True)` — one line of code. OTel buys cross-service span correlation, but the v1 system is single-process; correlation doesn't need SDK-level machinery.
- **WHY defer OTel to a v2/v3 trigger**: the migration cost when OTel arrives (multi-process eval sharding, or a distributed state store) is bounded — every emission site swaps from `logger.info(event=...)` to OTel spans. Paying that later is cheaper than running an OTel Collector daemon now for capability we don't consume.
- **WHY event categories that follow the agent loop boundaries** (`request.received`, `task_file.constructed`, `validation.passed/failed`, `tool.dispatch.start/end`, `recovery.triggered`, `clarification.emitted`, `response.emitted`): the trace tells the STORY of a request stage by stage. If a scenario fails at "step 3," the trace should map back to a specific loop transition, not just "something errored." Categories = the loop's discrete states from ADR 0001.
- **WHY redaction at emission time, not later**: one chokepoint that can't be bypassed by a forgotten downstream filter. If a new credential field is added next week and someone logs it, the emission-layer processor catches it before it hits disk. Doing redaction downstream means every consumer needs its own filter — inconsistent, error-prone, and one missing filter = leaked secret.
- **WHY ndjson vs plain-text logs**: unstructured logs are fast to write and miserable to query. Ronin's eval harness needs to programmatically filter and aggregate events; that requires structure. The marginal cost of JSON over text is essentially zero and the queryability difference is huge.

- **WHY ndjson vs database-backed events**: file-based ndjson is durable, append-only, trivially git-diffable, and trivially attachable to bug reports. A database couples telemetry to state-store lifecycle (rollbacks would lose trace data — the opposite of what you want during failure analysis).

Alternatives considered

Alternative A — OpenTelemetry from day 1

Adopt the OTel SDK with spans, attributes, and baggage; run an OTel Collector locally; export to Jaeger or Tempo for visualisation.

why rejected — v1 has zero need for cross-service span correlation (single-process agent + eval). The collector is an extra process to run, configure, and monitor. The OTel SDK adds a 30+ MB dependency for capability we don't use. We accept the migration cost later if v2/v3 introduces multi-process eval runs.

Alternative B — Plain text logs (stdlib `logging`)

Use Python's stdlib `logging` with a custom formatter; parse with grep / awk / regex when investigating failures.

why rejected — unstructured logs are fast to write and miserable to query. Ronin's eval harness needs to programmatically filter and aggregate trace events; that requires structure. The marginal cost of writing JSON instead of text is essentially zero.

Alternative C — Database-backed event log (SQLite events table)

Write events directly into the SQLite state store (ADR 0007) as rows in an `events` table.

why rejected — couples telemetry to the state-store lifecycle (rollbacks would lose trace data, which is the opposite of what you want during failure analysis). File-based jsonl is durable, append-only, and trivially backed up. Querying via `duckdb` or `pandas` is fast enough for v1 eval volumes (~100 scenarios × ~10 events each).

Consequences

Positive

- Zero new infrastructure to deploy or maintain in v1
- Trace files are append-only, trivially diffed, easy to attach to bug reports
- Ronin's eval harness reads jsonl with one line of pandas / polars
- Schema is forward-compatible: new event categories are additive
- Auth redaction at emission is a single chokepoint, easier to audit

Negative

- No standard tooling (Jaeger UI, etc.) for visualisation — Ronin's eval has to build its own analyses
- Migration cost when (if) we move to OTel in v2 — every emission site needs updating
- jsonl files in `traces/` will accumulate on disk; needs a retention policy by Phase 3

Neutral

- Event ordering relies on file-append order + timestamp; sufficient for single-process v1, would need explicit ordering if v3 introduces multi-process eval

---

Open questions / follow-ups

- **Trigger for OTel migration**: documented in DEFERRED.md. Likely Phase 3 if multi-process eval becomes useful.
- **Event sampling**: v1 captures every event. If volume becomes a problem at Phase 3, introduce per-event-category sampling rates.
- **Trace correlation across the agent → inference backend → adapter chain**: handled by passing `trace_id` through call signatures in v1; might become unwieldy at scale and motivate context-local storage.
- **Schema validation for events**: not enforced in v1 — emitters trust each other. Worth a JSON Schema for emission discipline if the event vocabulary expands.

References

- ndjson spec: https://github.com/ndjson/ndjson-spec
- OpenTelemetry data model: https://opentelemetry.io/docs/specs/otel/overview/
- structlog (used by ADR 0010): https://www.structlog.org
- RONIN_SYNC.txt Tier 1 #1 — open discussion

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.